RTL Design Sherpa

RTL AMBA AXI4-Stream Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXIS4 (AXI4-Stream) Modules

Location: `rtl/amba/axis4/` **Test Location:** `val/amba/` **Status:** Production Ready

---

1.1 Overview

The AXIS4 subsystem is a complete implementation of the ARM AMBA AXI4-Stream protocol — the streaming member of the AMBA family, built for high-performance unidirectional data flows. Video processing, network packet handling, DSP pipelines: anywhere data moves in one direction without an address in sight, this is what you reach for. AXI4-Stream gets its speed by throwing things away — no address channels, no burst bookkeeping — and what's left is a single channel that sustains one beat per cycle.

1.1.1 Key Features

- **AXI4-Stream Protocol:** Streaming-optimized subset (no address channels)
- **High Throughput:** One beat per cycle sustained; e.g. 25.6 GB/s at 512-bit / 400 MHz
- **Flexible Sideband Signals:** TID, TDEST, TUSER for routing/control
- **Packet Framing:** TLAST for packet/frame boundaries
- **Elastic Buffering:** Integrated skid buffers for timing closure
- **Clock Gating Variants:** Power-optimized versions
- **Typical Use:** Video streams, network packets, DSP pipelines

1.1.2 Module Categories

Two modules do the work — a master and a slave — and each has a clock-gated sibling for power-sensitive designs.

1.1.2.1 Core Master/Slave Modules

Module	Description	Documentation	Status
axis4_master	AXIS master with buffered output	axis4_master.md	Documented
axis4_slave	AXIS slave with buffered input	axis4_slave.md	Documented

1.1.2.2 Clock-Gated Variants

All clock-gated variants documented in: axis4_clock_gating_guide.md

Module	Base Module	Status
axis4_master_cg	axis4_master	Documented
axis4_slave_cg	axis4_slave	Documented

1.1.3 AXI4-Stream Protocol Overview

1.1.3.1 Streaming vs Memory-Mapped

If you already know AXI4, the mental model is short: AXI4-Stream is what remains when you strip the addressing off and let the data run.

Feature	AXI4	AXI4-Stream
Address Channels	AW/AR (5 channels)	None (streaming)
Data Flow	Bidirectional	Unidirectional
Transaction Type	Memory-mapped	Data streaming
Backpressure	Via ready signals	Via TREADY
Packet Framing	Not applicable	TLAST signal
Routing	Address-based	TID/TDEST-based
Typical Use	Register/memory access	Video/network/DSP

1.1.3.2 Signal Architecture

AXI4-Stream uses a single channel for streaming data:

Core Signals: - **TDATA** - Streaming data payload (8-512 bits) - **TSTRB** - Byte lane strobes (1 bit per byte) - **TVALID** - Data valid indicator - **TREADY** - Ready for data (backpressure) - **TLAST** - Last transfer in packet/frame

Optional Sideband Signals: - **TID** - Stream identifier (multiplexing) - **TDEST** - Destination routing - **TUSER** - User-defined control/status

Not implemented: ARM IHI 0051A also defines **TKEEP** (byte is part of the data stream, as distinct from **TSTRB**, which marks a data byte versus a position byte) and **TWAKEUP**. Neither has a port in these modules — only TSTRB is present. The buffer treats TSTRB as an opaque per-byte-lane field and carries it through unmodified, so a design may route a TKEEP mask over it as a local convention. That is not protocol-compliant TKEEP support: a receiver cannot distinguish null bytes from position bytes. Examples below that connect signals named `*_keep` to `*_tstrb` are relying on exactly that convention.

1.1.3.3 Key Signals

Data Transfer: - `TDATA[N:0]` - Primary streaming data - `TSTRB[N/8:0]` - Byte valid indicators (1=valid byte) - `TVALID` - Transfer valid - `TREADY` - Transfer ready

Packet Control: - TLAST - End of packet/frame marker

Routing/Control: - `TID[N:0]` - Stream ID for multiplexing (optional) - `TDEST[N:0]` - Destination routing information (optional) - `TUSER[N:0]` - User-defined metadata (optional)

1.2 Usage Examples

Four starting points cover most of what people build with these modules. Pick the one closest to your problem and adjust the widths.

1.2.1 Basic Stream Master

```
axis4_master #(
    .SKID_DEPTH(4),              // 4 entries
    .AXIS_DATA_WIDTH(64),        // 64 bits = 8 bytes per transfer
    .AXIS_ID_WIDTH(4),           // 16 streams
    .AXIS_DEST_WIDTH(4),         // 16 destinations
    .AXIS_USER_WIDTH(8)          // 8-bit control
) u_stream_master (
    .aclk                   (stream_clk),
    .aresetn                (stream_resetn),

    // Frontend interface (from source)
    .fub_axis_tdata         (src_tdata),
    .fub_axis_tstrb         (src_tstrb),
    .fub_axis_tlast         (src_tlast),
    .fub_axis_tid           (src_tid),
    .fub_axis_tdest         (src_tdest),
    .fub_axis_tuser         (src_tuser),
    .fub_axis_tvalid        (src_tvalid),
    .fub_axis_tready        (src_tready),

    // Master interface (to interconnect)
    .m_axis_tdata           (net_tdata),
    // ... rest of master signals

    .busy                   (master_busy)
);
```

1.2.2 Basic Stream Slave

```
axis4_slave #(
    .SKID_DEPTH(4),              // 4 entries
    .AXIS_DATA_WIDTH(64),
    .AXIS_ID_WIDTH(8),
    .AXIS_DEST_WIDTH(4),
```

```
    .AXIS_USER_WIDTH(1)
) u_stream_slave (
    .aclk               (stream_clk),
    .aresetn            (stream_resetn),

    // Slave interface (from interconnect)
    .s_axis_tdata       (net_tdata),
    .s_axis_tstrb       (net_tstrb),
    .s_axis_tlast       (net_tlast),
    .s_axis_tid         (net_tid),
    .s_axis_tdest       (net_tdest),
    .s_axis_tuser       (net_tuser),
    .s_axis_tvalid      (net_tvalid),
    .s_axis_tready      (net_tready),

    // Backend interface (to processor)
    .fub_axis_tdata     (proc_tdata),
    // ... rest of backend signals

    .busy               (slave_busy)
);
```

1.2.3 Video Processing Pipeline

```
// Video frame buffer with packet boundaries
axis4_master #(
    .SKID_DEPTH(4),           // 4-entry buffer
    .AXIS_DATA_WIDTH(96),     // 4 pixels x 24-bit RGB
    .AXIS_ID_WIDTH(0),        // No stream ID
    .AXIS_DEST_WIDTH(4),      // 16 display outputs
    .AXIS_USER_WIDTH(16)      // Frame/line metadata
) u_video_master (
    .aclk               (pixel_clk),
    .aresetn            (video_resetn),

    // From frame buffer
    .fub_axis_tdata     (pixel_data),      // RGB pixel data
    .fub_axis_tstrb     (pixel_strb),      // Valid bytes
    .fub_axis_tlast     (line_end),        // End of video line
    .fub_axis_tid       (1'b0),            // Unused
    .fub_axis_tdest     (display_id),      // Target display
    .fub_axis_tuser     ({frame_num, line_num}),
    .fub_axis_tvalid    (pixel_valid),
    .fub_axis_tready    (pixel_ready),

    // To video processing chain
    .m_axis_tdata       (proc_pixel_data),
    .m_axis_tstrb       (proc_pixel_strb),
```

```
    .m_axis_tlast       (proc_line_end),
    .m_axis_tid         (),
    .m_axis_tdest       (proc_display_id),
    .m_axis_tuser       (proc_metadata),
    .m_axis_tvalid      (proc_valid),
    .m_axis_tready      (proc_ready),

    .busy               (video_busy)
);
```

1.2.4 Network Packet Processing

```
// High-bandwidth packet processing
axis4_slave #(
    .SKID_DEPTH(6),           // 6-entry buffer for latency
    .AXIS_DATA_WIDTH(512),    // 64 bytes per beat
    .AXIS_ID_WIDTH(8),        // 256 flow IDs
    .AXIS_DEST_WIDTH(6),      // 64 output ports
    .AXIS_USER_WIDTH(16)      // Packet metadata
) u_packet_slave (
    .aclk               (net_clk),
    .aresetn            (net_resetn),

    // From network interface
    .s_axis_tdata       (rx_pkt_data),
    .s_axis_tstrb       (rx_pkt_keep),     // Byte valid
    .s_axis_tlast       (rx_pkt_eop),      // End of packet
    .s_axis_tid         (rx_flow_id),
    .s_axis_tdest       (rx_output_port),
    .s_axis_tuser       ({pkt_len, pkt_type}),
    .s_axis_tvalid      (rx_pkt_valid),
    .s_axis_tready      (rx_pkt_ready),

    // To packet processor
    .fub_axis_tdata     (proc_pkt_data),
    .fub_axis_tstrb     (proc_pkt_keep),
    .fub_axis_tlast     (proc_pkt_eop),
    .fub_axis_tid       (proc_flow_id),
    .fub_axis_tdest     (proc_port),
    .fub_axis_tuser     (proc_metadata),
    .fub_axis_tvalid    (proc_pkt_valid),
    .fub_axis_tready    (proc_pkt_ready),

    .busy               (packet_busy)
);
```

1.3 Design Notes

1.3.1 Design Patterns

1.3.1.1 Pattern 1: Elastic Buffering

All core modules use `gaxi_skid_buffer`: - Decouples source and sink timing - Configurable depth per module (2..8 inclusive entries) - 1-cycle latency overhead on **every** transfer — the buffer registers both data and `rd_valid`, so this is not a zero-bubble bypass skid - Full backpressure handling, one beat per cycle sustained once primed

That third bullet is the part that bites people. If your latency budget assumes an unstalled beat passes combinationally, it doesn't — not here.

1.3.1.2 Pattern 2: Packet Framing

TLAST signal marks packet boundaries:

```
// Video: TLAST marks end of line
tlast = (pixel_count == PIXELS_PER_LINE - 1);

// Network: TLAST marks end of packet
tlast = (byte_count == packet_length - 1);

// DSP: TLAST marks end of frame
tlast = (sample_count == FRAME_SIZE - 1);
```

1.3.1.3 Pattern 3: Stream Multiplexing

TID enables multiple logical streams:

```
// Four video streams on single physical interface
.fub_axis_tid(camera_id),  // 0-3 for 4 cameras
.fub_axis_tdest(display_id) // Route to specific display
```

1.3.1.4 Pattern 4: Clock Gating

Clock-gated variants (*_cg) add power management: - Dynamic gating driven by TVALID on either side, the downstream TREADY, and buffer occupancy - Runtime-configurable idle threshold (`cfg_cg_enable`, `cfg_cg_idle_count`) - 1 wake-up register stage; the first usable gated-clock edge arrives 2 cycles after activity, during which the incoming TREADY is held low - Best for packet-based streams with idle gaps

See the AXIS4 Clock-Gated Variants Guide for the exact wakeup terms and the ungating latency breakdown.

1.3.2 Performance Characteristics

The tables in this section are **design targets for scoping, not measured synthesis or power results.** No target device or technology node backs them; re-derive from your own synthesis run before committing to a system budget.

1.3.2.1 Throughput

Configuration	Bandwidth	Notes
32-bit @ 400 MHz	1.6 GB/s	Typical DSP
64-bit @ 400 MHz	3.2 GB/s	Video processing
128-bit @ 400 MHz	6.4 GB/s	High-bandwidth video
512-bit @ 400 MHz	25.6 GB/s	Network/storage

1.3.2.2 Latency

Path	Cycles	Notes
Buffer latency	1	Skid buffer overhead
Master → Slave	2-3	Through interconnect
Typical pipeline	5-10	Multi-stage processing

1.3.2.3 Resource Usage

Module Type	LUTs	FFs	BRAM	Notes
Master/Slave (64-bit)	~150	~100	0	Base module
Clock-gated (+cg)	not measured	+5	0	Counted from the RTL: `r_wakeup` (1 FF, `amba_clock_gate_ctrl`) + `r_idle_counter` (`IDLE_CNTR_WIDTH`, default 4, `clock_gate_`

Module Type	LUTs	FFs	BRAM	Notes
				`ctrl`). Scales as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG is a latch/BUFGCE, not a fabric flop
Wider data (512-bit)	~400	~350	0	8x data width

vs AXI4: an estimated 60-70% resource saving, attributable to the absence of the five address/response channels and of any outstanding-transaction tracking. No AXI4 baseline measurement is published in this document, so treat the figure as a rough expectation rather than a benchmarked result.

1.3.3 Common Use Cases

1.3.3.1 Video Processing

Characteristics: - High throughput (1-10 GB/s) - Regular packet structure (lines/frames) - TLAST marks line/frame boundaries - TUSER carries frame metadata

Recommended Configuration:

```
.AXIS_DATA_WIDTH(64-128),  // Multiple pixels per cycle
.SKID_DEPTH(4),            // 4-entry buffer
.AXIS_USER_WIDTH(8-16)     // Frame/line metadata
```

1.3.3.2 Network Packet Processing

Characteristics: - Variable packet sizes - High packet rate - TID for flow identification - TDEST for output routing

Recommended Configuration:

```
.AXIS_DATA_WIDTH(256-512), // Wide data bus
.SKID_DEPTH(6),            // 6-entry buffer
.AXIS_ID_WIDTH(8),         // 256 flows
.AXIS_DEST_WIDTH(6)        // 64 ports
```

1.3.3.3 DSP Pipelines

Characteristics: - Continuous data flow - Fixed sample rate - Minimal sideband signals - Low latency requirements

Recommended Configuration:

```
.AXIS_DATA_WIDTH(32-128),  // Sample data
.SKID_DEPTH(2),            // Minimal buffer
.AXIS_ID_WIDTH(0),         // No multiplexing
.AXIS_DEST_WIDTH(0),       // No routing
.AXIS_USER_WIDTH(4)        // Minimal metadata
```

1.3.4 Parameter Selection Guidelines

1.3.4.1 Data Width (AXIS_DATA_WIDTH)

Application	Recommended Width	Rationale
Audio processing	32-64 bits	1-2 samples per cycle
SD video	64-96 bits	2-4 pixels per cycle
HD video	128-256 bits	Multiple pixels per cycle
4K video	256-512 bits	High pixel throughput
Network (1G)	64-128 bits	Moderate packet rate
Network (10G/100G)	256-512 bits	High packet rate

1.3.4.2 Buffer Depth (SKID_DEPTH)

`SKID_DEPTH` is a **literal entry count**, not a log2 exponent, and is passed straight to `gaxi_skid_buffer.DEPTH`. Only the values 2..8 inclusive are supported.

SKID_DEPTH	Buffer Size	Use Case
2	2 entries	Low latency, continuous streams
4	4 entries	Video processing, typical DSP pipelines (default)
6	6 entries	Network packets, variable latency
8	8 entries	Maximum decoupling available

The skid buffer is a timing element, not a rate-adaptation FIFO. If an application needs tens or hundreds of entries of elastic storage, place a `gaxi_fifo_sync` (or `gaxi_fifo_async` across clock domains) downstream instead of scaling `SKID_DEPTH`.

1.3.4.3 Sideband Signal Widths

TID (Stream ID): - 0: Single stream, no multiplexing - 4: Up to 16 concurrent streams - 8: Up to 256 concurrent streams

TDEST (Destination): - 0: No routing (point-to-point) - 4: Up to 16 destinations - 6: Up to 64 destinations (network)

TUSER (User Metadata): - 0-1: Minimal or no metadata - 4-8: Frame/packet control - 16+: Complex metadata

1.3.5 Known Limitations

Applies to `axis4_master`, `axis4_slave`, and their `_cg` variants:

Limitation	Detail
No `TKEEP`	Only `TSTRB` is implemented. Null bytes cannot be distinguished from position bytes
No `TWAKEUP`	The AXI4-Stream low-power handshake signal is not implemented
No native CDC	Both interfaces of every module are on `aclk`. Crossing clock domains requires an external `gaxi_fifo_async`
No routing, arbitration, or multiplexing	`TID`/`TDEST` are carried through unmodified, never decoded. No `axis_arbiter` or `axis_interconnect` module exists in this repository
No protocol checking	These modules do not detect or report `TVALID` deassertion before `TREADY`, or `TLAST` framing errors
No monitor bus output	Unlike the AXI4/AXIL4/APB monitors, the AXIS4 modules emit no monbus packets. For stream instrumentation see

Limitation	Detail
	axis_bus_meter
`SKID_DEPTH` limited to 2..8 inclusive	The skid buffer is a timing element, not a rate adapter
1 register stage / 2-cycle ungating latency (_cg)	The first beat after an idle period is backpressured while the clock restarts

1.3.6 When to Use AXI4-Stream

Use AXI4-Stream For: - Video/image processing pipelines - Network packet processing - DSP data flows - High-throughput unidirectional data - Streaming accelerators

Use AXI4 For: - Memory-mapped register access - Random access patterns - Bidirectional data flows - Address-based routing

1.3.7 Buffer Depth Trade-offs

Shallow Buffers (SKID_DEPTH = 2): - Lower latency - Smaller area - Less tolerance for backpressure - **Use for:** Low-latency DSP, continuous streams

Deep Buffers (SKID_DEPTH = 6-8): - High backpressure tolerance - Better throughput under variable load - Higher latency - Larger area - **Use for:** Network packets, variable-latency paths

No single depth wins everywhere — you're trading latency and area against backpressure tolerance, so size for the stall pattern you actually expect.

1.3.8 Sideband Signal Optimization

Minimize unused signals for area/power:

```
// Video processing - no routing needed
.AXIS_ID_WIDTH(0),     // No stream multiplexing
.AXIS_DEST_WIDTH(0),   // No destination routing
.AXIS_USER_WIDTH(8)    // Only frame metadata

// vs Network processing - full routing
.AXIS_ID_WIDTH(8),     // 256 flows
.AXIS_DEST_WIDTH(6),   // 64 ports
.AXIS_USER_WIDTH(16)   // Full packet metadata
```

1.4 Related Modules

1.4.1 Protocol Specifications

- ARM IHI 0051A: AMBA AXI4-Stream Protocol Specification

1.4.2 RTL Design Sherpa Documentation

- **rtl-amba Overview** - Complete AMBA subsystem architecture
- **AXI4 Modules** - Memory-mapped protocol (comparison)
- **AXIL4 Modules** - Simplified control interface
- **GAXI Modules** - Generic AXI utilities (buffers, FIFOs)

1.4.3 Source Code

- RTL: `rtl/amba/axis4/`
- Tests: `val/amba/test_axis*.py`
- Framework: `bin/TBClasses/axis4/`
- Shared Infrastructure: `rtl/amba/shared/` (gaxi_skid_buffer)

1.5 Testing

All AXIS4 modules are verified using CocoTB-based testbenches:

```
# Run all AXIS4 tests
pytest val/amba/test_axis*.py -v

# Run specific module tests
pytest val/amba/test_axis4_master.py -v
pytest val/amba/test_axis4_slave.py -v

# Run clock-gated variant tests
pytest val/amba/test_axis4_master_cg.py -v
pytest val/amba/test_axis4_slave_cg.py -v

# Run with waveforms
pytest val/amba/test_axis4_master.py --vcd=waves.vcd -v
```

Last Updated: 2025-10-20

1.6 Navigation

- ← **Back to rtl-amba Index**
- ← **Back to Main Documentation Index**

2 AXIS4 Clock-Gated Variants Guide

Location: `rtl/amba/axis4/*_cg.sv` **Status:** Production Ready

2.1 Overview

Both AXIS4 modules have clock-gated (`_cg`) variants that add power management through dynamic clock gating. Same architecture as AXI4/AXIL4 clock gating, optimized for streaming protocols. The deal is simple: when the stream goes quiet, the clock stops; when traffic shows up, the clock restarts and the beat moves.

2.1.1 Available Clock-Gated Modules

Module	Base Module	Description
`axis4_master_cg`	axis4_master	Clock-gated stream master
`axis4_slave_cg`	axis4_slave	Clock-gated stream slave

2.1.2 Key Features

- **Dynamic Clock Gating:** Automatic clock disable during idle
- **Configurable Idle Threshold:** Programmable idle count before gating
- **Functional Equivalence:** Identical data behaviour to the base modules once ungated
- **Status Monitoring:** Real-time `cg_gating` and `cg_idle` indication
- **Low Performance Impact:** A short ungating latency on the first beat after an idle period (see Clock Gating Behavior)

No scan-bypass port. These wrappers have no `cg_test_enable` or equivalent test input. For scan/DFT, hold `cfg_cg_enable = 0`, which forces the ICG permanently enabled and makes the wrapper functionally identical to the base module.

2.2 Parameters

The _cg variants add exactly one parameter to the base module's list:

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = 2^N-1 cycles)

All other parameters identical to base module.

2.3 Ports

2.3.1 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating (0=disabled, 1=enabled)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

2.3.2 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	Clock currently gated (1=gated, 0=running)
cg_idle	1	Output	No activity was observed in the previous cycle (registered ~wakeup)

> **The _cg wrappers do not expose busy.** The base module's busy output is consumed internally as one of the wakeup terms and is not brought out. Use cg_idle for system-level power sequencing instead. Apart from that substitution, and the two cfg_cg_* inputs above, the port list matches the base module.
>
> There is **no cg_clk_count** port. Cumulative gated-cycle counting is not implemented in these wrappers.

2.4 Functional Description

2.4.1 Clock Gating Behavior

Both wrappers build a single wakeup term and hand it to `amba_clock_gate_ctrl`. For `axis4_master_cg` that term is:

```
user_valid = fub_axis_tvalid || busy;   // busy is internal;
m_axis_tvalid is in axi_valid
axi_valid  = m_axis_tvalid;
wakeup     = user_valid || axi_valid;                    // registered
one cycle
```

> **A peer's READY must never appear in the activity term.** A consumer that parks its response-ready high while idle is behaving correctly; folding that in pins the block permanently awake and defeats gating entirely, silently, because function is unaffected. Use the VALID your side drives. Ten `_cg` wrappers carried this defect until 2026-09-02.

`axis4_slave_cg` uses the same structure with `s_axis_tvalid`, `fub_axis_tready` and `fub_axis_tvalid` substituted. Note that the **sink's ready signal is a wakeup term**: a downstream block holding TREADY high keeps the clock running even with no traffic.

Gating Conditions (All Must Be True): 1. No activity on any of the wakeup terms above 2. Idle countdown has expired — gating engages `cfg_cg_idle_count + 1` cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count + 2` cycles after the last bus activity on AXI4-Stream (one extra for the `r_wakeup` flop) 3. Gating enabled (`cfg_cg_enable = 1`)

Ungating Conditions (Any Triggers Ungating): 1. Any wakeup term asserts (TVALID on either side, or a non-empty buffer). A peer's TREADY is NOT a wakeup term and must never be one – see `vault/handbook/design/clock-gating-activity-terms.md`. 2. `cfg_cg_enable` deasserted

---

2.5 Timing Characteristics

2.5.1 Ungating Latency

Ungating is not instantaneous. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. The AXIS4 wrappers drive `user_valid`/`axi_valid` combinationally, so they sit in the single-stage group:

Stage	Cost	Source
wakeup register in `amba_clock_gate_ctrl`	1 cycle	`r_wakeup <= user_valid || axi_valid`

Stage	Cost	Source
ICG enable in `clock_gate_ctrl`	0 cycles (combinational)	`w_gate_enable = cfg_cg_enable && !wakeup && (r_idle_counter == 'h0)`
First usable gated-clock rising edge	1 cycle	The released clock is only usable on the following edge

1 register stage; the first usable gated-clock edge arrives 2 `aclk` cycles after activity asserts. The `clock_gate_ctrl` header comment “Wakeup: 1 clock from wakeup assertion to clock restoration” refers to that resulting edge, not to a register stage.

> The AXI5-Stream (`axis5_*_cg`) wrappers register activity locally before handing it to `amba_clock_gate_ctrl`, so they are a two-stage design: 3 cycles to the first usable edge. See axis5_master_cg.

This latency is not free of protocol impact. Both wrappers force the incoming ready signal low while gated:

```
// axis4_master_cg
assign fub_axis_tready = cg_gating ? 1'b0 : int_tready;
// axis4_slave_cg
assign s_axis_tready   = cg_gating ? 1'b0 : int_tready;
```

So the first beat presented after an idle period is **backpressured for the ungating latency** rather than accepted immediately. Budget this stall when sizing `cfg_cg_idle_count` for latency-sensitive streams: an aggressive idle count trades first-beat latency for power.

2.6 Usage Examples

```
axis4_master_cg #(
    // Base parameters
    .SKID_DEPTH(4),
    .AXIS_DATA_WIDTH(64),
    .AXIS_ID_WIDTH(8),
    .AXIS_DEST_WIDTH(4),
    .AXIS_USER_WIDTH(1),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH(4)  // Up to 15 idle cycles
) u_axis_master_cg (
    .aclk(stream_clk),
    .aresetn(stream_resetn),
```

```
    // Clock gating configuration
    .cfg_cg_enable(1'b1),          // Enable gating
    .cfg_cg_idle_count(4'd3),      // Gate after 3 idle cycles

    // AXIS signals (identical to base module)
    .fub_axis_tdata(src_tdata),
    // ... rest of AXIS signals ...

    // Clock gating status
    .cg_gating(stream_clk_gated),
    .cg_idle(stream_idle)
);
```

2.7 Design Notes

2.7.1 Configuration Guidelines

2.7.1.1 Idle Count Selection for Streaming

Aggressive Gating (Burst Packets):

```
.cfg_cg_idle_count(4'd1)   // Gate after 1 idle cycle
// Use for: Packet networks with gaps between packets
```

Moderate Gating (Mixed Traffic):

```
.cfg_cg_idle_count(4'd5)   // Gate after 5 idle cycles
// Use for: Video processing with blanking periods
```

Conservative Gating (Continuous Streams):

```
.cfg_cg_idle_count(4'd10)  // Gate after 10 idle cycles
// Use for: Low-latency continuous data flows
```

Disable Gating:

```
.cfg_cg_enable(1'b0)
// Use for: 100% utilization continuous streams
```

2.7.2 When to Use Clock Gating

Recommended For: - Packet-based streaming (idle between packets) - Video processing (blanking periods between frames/lines) - Burst data transfers (gaps between bursts) - Power-constrained streaming applications

Not Recommended For: - Continuous audio/video streams (100% utilization) - Real-time DSP pipelines (no idle periods) - Ultra-low-latency paths

The pattern there is honest: gating pays when your stream has gaps. A stream that never idles gives the gate nothing to work with.

2.7.3 Power Savings Estimates

The figures below are **estimates for planning purposes, not measured power results.** They have not been correlated against a power analysis run on any target technology.

Traffic Pattern	Duty Cycle	Idle Count	Estimated Savings
Sporadic control stream	5%	1	60-70%
Burst transfers	30%	1	35-40%
Packet network (1500B packets)	40%	1	30-35%
Video (1080p with blanking)	70%	5	10-15%
Continuous stream	100%	any	0% (set `cfg_cg_enable = 0`)

Notes: - Savings apply to the gated clock tree and the sequential elements downstream of it. The `amba_clock_gate_ctrl` logic itself, and the wakeup-term combinational cone, remain ungated and consume power at all duty cycles. - Streaming typically has longer continuous active periods than register access, resulting in lower power savings than AXI4-Lite. - At 100% duty cycle, gating never engages, so the wrapper is pure overhead. Instantiate the base module instead, or tie `cfg_cg_enable` low.

AXIS-specific notes: the architecture is the same as AXI4/AXIL4 clock gating, power savings vary based on stream duty cycle, and the best fit is packet-based or frame-based streams.

2.8 Related Modules

For full clock gating details, see: - AXI4 Clock Gating Guide - Complete reference - AXIL4 Clock Gating Guide - Additional examples

2.8.1 Base Modules

- axis4_master - Base stream master

- axis4_slave - Base stream slave

2.8.2 Architecture

- AXI4 Clock Gating Guide - Complete reference
- AXIS4 Index - AXIS4 module index

Last Updated: 2025-10-20

2.9 Navigation

- ← Back to AXIS4 Index
- ← Back to rtl-amba Index

3 axis4_master

An AXI4-Stream master module that provides high-throughput streaming data transmission with configurable buffering and comprehensive support for all AXI4-Stream sideband signals including ID, DEST, and USER channels.

3.1 Overview

The `axis4_master` module implements a complete AXI4-Stream master interface with integrated skid buffering for optimal streaming performance. It supports the full AXI4-Stream protocol with configurable data widths, optional sideband signals, and intelligent buffer management to maximize throughput in streaming data applications such as video processing, network packet handling, and DSP pipelines. In practice it's a shallow elastic buffer with an AXIS face on each side — which is exactly what you want between a producer and an interconnect that don't quite agree on timing.

3.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Skid buffer depth in **entries** (not a log2 exponent). Passed directly to `gaxi_skid_buffer.DEPTH`, which supports 2..8 inclusive

Parameter	Type	Default	Description
AXIS_DATA_WIDTH	int	32	AXI4-Stream data bus width in **bits** (must be a multiple of 8; `SW = AXIS_DATA_WIDTH/8`)
AXIS_ID_WIDTH	int	8	Stream ID width (0 to disable)
AXIS_DEST_WIDTH	int	4	Destination width (0 to disable)
AXIS_USER_WIDTH	int	1	User signal width (0 to disable)

> **SKID_DEPTH is a literal entry count.** `SKID_DEPTH = 4` yields a 4-entry buffer, not 16. The underlying `gaxi_skid_buffer` is a shift-register FIFO whose `count` port is 4 bits wide; only the values 2..8 inclusive are supported. Any integer in that range is legal, odd values included – `gaxi_skid_buffer` states "2..8 inclusive (any integer)" and guards it. The `{2,4,6,8}` restriction claimed here was an inference, not a contract.

3.3 Ports

The full declaration, straight from the RTL:

```
module axis4_master #(
    parameter int SKID_DEPTH          = 4,
    parameter int AXIS_DATA_WIDTH     = 32,
    parameter int AXIS_ID_WIDTH       = 8,
    parameter int AXIS_DEST_WIDTH     = 4,
    parameter int AXIS_USER_WIDTH     = 1,

    // Short and calculated params
    parameter int DW        = AXIS_DATA_WIDTH,
    parameter int IW        = AXIS_ID_WIDTH,
    parameter int DESTW     = AXIS_DEST_WIDTH,
    parameter int UW        = AXIS_USER_WIDTH,
    parameter int SW        = DW / 8,
    parameter int IW_WIDTH = (IW > 0) ? IW : 1,     // Minimum 1 bit
for zero-width signals
    parameter int DESTW_WIDTH = (DESTW > 0) ? DESTW : 1,
    parameter int UW_WIDTH = (UW > 0) ? UW : 1,
    parameter int TSize     = DW+SW+1+IW_WIDTH+DESTW_WIDTH+UW_WIDTH  //
Total packet size
) (
```

```
    // Global Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,

    // Slave AXI4-Stream Interface (Input Side - FUB)
    input  logic [DW-1:0]               fub_axis_tdata,
    input  logic [SW-1:0]               fub_axis_tstrb,
    input  logic                        fub_axis_tlast,
    input  logic [IW_WIDTH-1:0]         fub_axis_tid,
    input  logic [DESTW_WIDTH-1:0]      fub_axis_tdest,
    input  logic [UW_WIDTH-1:0]         fub_axis_tuser,
    input  logic                        fub_axis_tvalid,
    output logic                        fub_axis_tready,

    // Master AXI4-Stream Interface (Output Side)
    output logic [DW-1:0]               m_axis_tdata,
    output logic [SW-1:0]               m_axis_tstrb,
    output logic                        m_axis_tlast,
    output logic [IW_WIDTH-1:0]         m_axis_tid,
    output logic [DESTW_WIDTH-1:0]      m_axis_tdest,
    output logic [UW_WIDTH-1:0]         m_axis_tuser,
    output logic                        m_axis_tvalid,
    input  logic                        m_axis_tready,

    // Status outputs for clock gating
    output logic                        busy
);
```

3.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI4-Stream clock
aresetn	1	Input	AXI4-Stream active-low reset

3.3.2 Slave AXI4-Stream Interface (Input Side - FUB)

Port	Width	Direction	Description
fub_axis_tdata	AXIS_DATA_WIDTH	Input	Stream data
fub_axis_tstrb	AXIS_DATA_WIDTH/8	Input	Data byte strobes

Port	Width	Direction	Description
fub_axis_tlast	1	Input	Last transfer in packet
fub_axis_tid	AXIS_ID_WIDTH	Input	Stream identifier
fub_axis_tdest	AXIS_DEST_WIDTH	Input	Destination routing
fub_axis_tuser	AXIS_USER_WIDTH	Input	User-defined sideband
fub_axis_tvalid	1	Input	Transfer valid
fub_axis_tready	1	Output	Transfer ready

3.3.3 Master AXI4-Stream Interface (Output Side)

Port	Width	Direction	Description
m_axis_tdata	AXIS_DATA_WIDTH	Output	Stream data
m_axis_tstrb	AXIS_DATA_WIDTH/8	Output	Data byte strobes
m_axis_tlast	1	Output	Last transfer in packet
m_axis_tid	AXIS_ID_WIDTH	Output	Stream identifier
m_axis_tdest	AXIS_DEST_WIDTH	Output	Destination routing
m_axis_tuser	AXIS_USER_WIDTH	Output	User-defined sideband
m_axis_tvalid	1	Output	Transfer valid
m_axis_tready	1	Input	Transfer ready

3.3.4 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for

Port	Width	Direction	Description
			clock gating

3.4 Functional Description

3.4.1 Skid Buffer Design

The module employs a single GAXI skid buffer to manage streaming data flow:

1. **Input Buffering**: Incoming stream data buffered for flow control
2. **Packet Packing**: All stream signals packed into single buffer entry
3. **Flexible Unpacking**: Conditional unpacking based on enabled sideband signals

3.4.2 Data Flow

```
FUB AXIS → Skid Buffer → Master AXIS → Downstream
     ↑            ↓               ↓
    Ready     Flow Control     Ready/Valid
```

3.4.3 Key Features

- **AXI4-Stream Compliance**: Full protocol support including all optional signals
- **Flexible Configuration**: Zero-width support for unused sideband signals
- **Flow Control**: Skid buffer prevents pipeline stalls
- **Clock Gating Support**: Busy signal for power optimization
- **Packet Integrity**: TLAST preservation through buffering
- **Multi-Stream Support**: TID and TDEST routing capabilities

3.4.4 Buffer Management

The skid buffer provides: 1. **Decoupling**: Separates upstream and downstream timing 2. **Flow Control**: Prevents data loss during backpressure 3. **Pipeline Optimization**: Eliminates ready-path combinatorial logic

3.4.5 Conditional Signal Handling

The module uses generate blocks to handle various combinations of enabled/disabled sideband signals:

```
// Example: Full signals enabled
if (IW > 0 && DESTW > 0 && UW > 0) begin
    // All sideband signals active
end else if (IW > 0 && DESTW == 0 && UW == 0) begin
```

```
    // Only TID active
end
// ... additional combinations
```

3.4.6 Busy Signal Generation

Activity detection for clock gating:

```
busy = (buffer_count > 0) || fub_axis_tvalid;
```

3.4.7 Signal Description

3.4.7.1 Core Signals

Signal	Width	Description
TDATA	8-512 bits	Primary data payload
TSTRB	TDATA/8 bits	Byte lane strobes, one bit per byte lane
TVALID	1 bit	Transfer valid indicator
TREADY	1 bit	Transfer ready (backpressure)
TLAST	1 bit	Last transfer in packet/frame

3.4.7.2 TSTRB and TKEEP

ARM IHI 0051A defines two byte qualifiers: TKEEP (byte is part of the data stream) and TSTRB (byte is a data byte rather than a position byte). This module implements **TSTRB only** — there is no TKEEP port.

The buffer is byte-qualifier agnostic: `fub_axis_tstrb` is packed into the skid buffer entry and reproduced on `m_axis_tstrb` unmodified, so it can equally carry a TKEEP mask if the surrounding design treats it that way. Doing so is a naming convention on the integrator's side, not protocol-compliant TKEEP support. See Known Limitations.

3.4.7.3 Optional Sideband Signals

Signal	Width	Description
TID	0-16 bits	Stream identifier for multiplexing
TDEST	0-16 bits	Destination routing information

Signal	Width	Description
TUSER	0-16 bits	User-defined control/status

3.5 Timing Characteristics

3.5.1 Buffer Performance

Characteristic	Value	Description
Buffer Depth	4 entries (default)	`SKID_DEPTH` entries, verbatim
Buffering Latency	1 clock cycle	Input to output delay
Flow Control Latency	1 clock cycle	Ready propagation

`gaxi_skid_buffer` registers both `rd_valid` and the storage array, so the 1-cycle input-to-output latency applies on **every** transfer, including the unstalled case. This is not a bypass ("zero-bubble") skid buffer; there is no combinational path from `fub_axis_tdata` to `m_axis_tdata`. Full throughput (one beat per cycle) is still sustained once the pipeline is primed.

3.5.2 Throughput Metrics

The figures below are **design targets for scoping purposes, not measured synthesis results.** No target device, technology node, or timing report backs them. Treat them as order-of-magnitude guidance and re-derive from your own synthesis run before committing to a system budget.

Metric	Indicative Value	Conditions
Maximum Frequency	400-800 MHz	Technology dependent; unqualified by node
Peak Throughput	3.2-25.6 GB/s	64-bit to 512-bit at the frequencies above
Sustained Throughput	95-99% of peak	With proper buffering
Pipeline Efficiency	>95%	Continuous data flow

3.6 Usage Examples

> **Notation:** some examples below abbreviate a full port list as `.m_axis_*(name_*)` or `.fub_axis_*(iface.*)`. **This is shorthand for the**

reader, not legal SystemVerilog — a port-name wildcard of that form does not compile. Expand it to explicit named connections (as in the "Basic Video Stream Processing" example) or use an interface port. The examples using this shorthand are illustrative of topology only and are not drop-in compilable.

3.6.1 Basic Video Stream Processing

```
axis4_master #(
    .SKID_DEPTH(4),              // 4-entry buffer
    .AXIS_DATA_WIDTH(64),        // 8 bytes per transfer
    .AXIS_ID_WIDTH(4),           // Support 16 streams
    .AXIS_DEST_WIDTH(4),         // Support 16 destinations
    .AXIS_USER_WIDTH(8)          // 8-bit control signals
) u_video_stream (
    .aclk            (video_clk),
    .aresetn         (video_resetn),

    // Input from video source
    .fub_axis_tdata    (pixel_data),
    .fub_axis_tstrb    (pixel_strb),
    .fub_axis_tlast    (line_end),
    .fub_axis_tid      (stream_id),
    .fub_axis_tdest    (display_id),
    .fub_axis_tuser    (pixel_ctrl),
    .fub_axis_tvalid   (pixel_valid),
    .fub_axis_tready   (pixel_ready),

    // Output to video pipeline
    .m_axis_tdata      (pipe_tdata),
    .m_axis_tstrb      (pipe_tstrb),
    .m_axis_tlast      (pipe_tlast),
    .m_axis_tid        (pipe_tid),
    .m_axis_tdest      (pipe_tdest),
    .m_axis_tuser      (pipe_tuser),
    .m_axis_tvalid     (pipe_tvalid),
    .m_axis_tready     (pipe_tready),

    .busy              (video_busy)
);
```

3.6.2 Network Packet Processing

```
// High-performance packet processing
axis4_master #(
    .SKID_DEPTH(8),              // 8-entry buffer (deepest legal) for
latency tolerance
```

```
    .AXIS_DATA_WIDTH(512),    // 64 bytes per beat (512-bit)
    .AXIS_ID_WIDTH(8),        // 256 flow IDs
    .AXIS_DEST_WIDTH(6),      // 64 output ports
    .AXIS_USER_WIDTH(16)      // Packet metadata
) u_packet_master (
    .aclk            (net_clk),
    .aresetn         (net_resetn),

    // Input from packet classifier
    .fub_axis_tdata    (pkt_data),
    .fub_axis_tstrb    (pkt_keep),
    .fub_axis_tlast    (pkt_eop),
    .fub_axis_tid      (flow_id),
    .fub_axis_tdest    (output_port),
    .fub_axis_tuser    ({pkt_len, pkt_type}),
    .fub_axis_tvalid   (pkt_valid),
    .fub_axis_tready   (pkt_ready),

    // Output to switching fabric
    .m_axis_*(switch_axis_*),

    .busy              (pkt_proc_busy)
);
```

3.6.3 DSP Data Pipeline

```
// DSP processing chain with minimal sideband
axis4_master #(
    .SKID_DEPTH(2),           // 2-entry buffer (minimum latency)
    .AXIS_DATA_WIDTH(128),    // 4 x 32-bit samples
    .AXIS_ID_WIDTH(0),        // No stream ID needed
    .AXIS_DEST_WIDTH(0),      // No destination routing
    .AXIS_USER_WIDTH(4)       // Sample metadata
) u_dsp_stream (
    .aclk            (dsp_clk),
    .aresetn         (dsp_resetn),

    // Input from ADC interface
    .fub_axis_tdata    (adc_samples),
    .fub_axis_tstrb    (adc_valid_bytes),
    .fub_axis_tlast    (frame_end),
    .fub_axis_tid      (1'b0),          // Unused
    .fub_axis_tdest    (1'b0),          // Unused
    .fub_axis_tuser    (sample_metadata),
    .fub_axis_tvalid   (adc_valid),
    .fub_axis_tready   (adc_ready),

    // Output to DSP processing
```

```
    .m_axis_tdata      (proc_samples),
    .m_axis_tstrb      (proc_strb),
    .m_axis_tlast      (proc_frame_end),
    .m_axis_tid        (),               // Unconnected
    .m_axis_tdest      (),               // Unconnected
    .m_axis_tuser      (proc_metadata),
    .m_axis_tvalid     (proc_valid),
    .m_axis_tready     (proc_ready),

    .busy              (dsp_busy)
);
```

3.6.4 Multi-Stream Router Integration

```
module stream_router (
    input logic axi_clk,
    input logic axi_resetn,

    // Multiple input streams
    axi4s_if.slave  input_streams [4],
    // Single output stream
    axi4s_if.master output_stream
);

    // Stream masters for each input with buffering
    genvar i;
    generate
        for (i = 0; i < 4; i++) begin : gen_stream_masters
            axis4_master #(
                .SKID_DEPTH(4),
                .AXIS_DATA_WIDTH(64),
                .AXIS_ID_WIDTH(4),
                .AXIS_DEST_WIDTH(4),
                .AXIS_USER_WIDTH(1)
            ) u_stream_master (
                .aclk(axi_clk),
                .aresetn(axi_resetn),

                // Connect to input stream
                .fub_axis_*(input_streams[i].*),

                // Connect to arbiter
                .m_axis_*(arb_inputs[i].*),

                .busy(stream_busy[i])
            );
        end
```

```
    endgenerate

    // Stream arbiter for output selection
    axis_arbiter #(
        .NUM_INPUTS(4),
        .DATA_WIDTH(64)
    ) u_arbiter (
        .aclk(axi_clk),
        .aresetn(axi_resetn),
        .s_axis(arb_inputs),
        .m_axis(output_stream),
        .active_mask(stream_enable)
    );

endmodule
```

3.6.5 Advanced Integration Patterns

3.6.5.1 Clock Domain Crossing

```
// Cross clock domains with async FIFO
module axis_cdc_system (
    // Source domain
    input  logic        src_clk,
    input  logic        src_resetn,
    axi4s_if.slave      src_axis,

    // Destination domain
    input  logic        dst_clk,
    input  logic        dst_resetn,
    axi4s_if.master     dst_axis
);

    // Source domain buffering
    axis4_master #(
        .SKID_DEPTH(2),
        .AXIS_DATA_WIDTH(32),
        .AXIS_ID_WIDTH(4),
        .AXIS_DEST_WIDTH(4),
        .AXIS_USER_WIDTH(1)
    ) u_src_master (
        .aclk(src_clk),
        .aresetn(src_resetn),
        .fub_axis_*(src_axis.*),
        .m_axis_*(cdc_src_axis.*),
        .busy(src_busy)
    );
```

```
    // Async clock domain crossing.
    // gaxi_fifo_async.DEPTH is also a literal entry count (default
16).
    // DATA_WIDTH must equal the packed TSize of the stream being
carried:
    //   TSize = DW + DW/8 + 1 + IW_WIDTH + DESTW_WIDTH + UW_WIDTH
    gaxi_fifo_async #(
        .DEPTH(64),                        // 64 entries
        .DATA_WIDTH(32 + 4 + 1 + 4 + 4 + 1)  // DW=32, SW=4, TLAST,
TID=4, TDEST=4, TUSER=1
    ) u_cdc_fifo (
        .axi_wr_aclk(src_clk),
        .axi_wr_aresetn(src_resetn),
        .wr_valid(cdc_src_tvalid),
        .wr_ready(cdc_src_tready),
        .wr_data(cdc_src_packed),      // the packed TSize word, not a
bundle

        .axi_rd_aclk(dst_clk),
        .axi_rd_aresetn(dst_resetn),
        .rd_valid(dst_tvalid),
        .rd_ready(dst_tready),
        .rd_data(dst_packed)
    );

endmodule
```

3.6.5.2 Stream Processing Pipeline

```
// Multi-stage processing pipeline
module stream_pipeline (
    input logic clk, resetn,
    axi4s_if.slave  input_stream,
    axi4s_if.master output_stream
);

    // Pipeline stage interfaces
    axi4s_if stage1_out();
    axi4s_if stage2_out();
    axi4s_if stage3_out();

    // Stage 1: Input buffering
    axis4_master #(.SKID_DEPTH(2), .AXIS_DATA_WIDTH(64))
    u_stage1 (
        .aclk(clk), .aresetn(resetn),
        .fub_axis_*(input_stream.*),
        .m_axis_*(stage1_out.*),
        .busy(stage1_busy)
```

```
    );

    // Stage 2: Processing with buffering
    stream_processor u_processor (
        .clk(clk), .resetn(resetn),
        .s_axis(stage1_out), .m_axis(stage2_out)
    );

    axis4_master #(.SKID_DEPTH(4), .AXIS_DATA_WIDTH(64))
    u_stage2 (
        .aclk(clk), .aresetn(resetn),
        .fub_axis_*(stage2_out.*),
        .m_axis_*(stage3_out.*),
        .busy(stage2_busy)
    );

    // Stage 3: Output conditioning
    axis4_master #(.SKID_DEPTH(2), .AXIS_DATA_WIDTH(64))
    u_stage3 (
        .aclk(clk), .aresetn(resetn),
        .fub_axis_*(stage3_out.*),
        .m_axis_*(output_stream.*),
        .busy(stage3_busy)
    );

    assign pipeline_busy = stage1_busy | stage2_busy | stage3_busy;

endmodule
```

3.6.5.3 Clock Gating Integration

The clock-gated variant `axis4_master_cg` wraps this module and drives it from a gated clock. Its control ports are `cfg_cg_enable` and `cfg_cg_idle_count`; its status ports are `cg_gating` and `cg_idle`. There is **no `cg_enable`, no `cg_test_enable`, and no busy output** on the `_cg` wrapper — the base module's busy is consumed internally as one of the wakeup terms.

```
// Clock gated version for power optimization
axis4_master_cg #(
    .SKID_DEPTH(4),
    .AXIS_DATA_WIDTH(64),
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg_master (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Clock gating configuration
    .cfg_cg_enable      (stream_cg_enable),
```

```
    .cfg_cg_idle_count  (4'd8),

    // Standard AXI4-Stream interfaces (expand to explicit
connections)
    // .fub_axis_tdata (...), ... , .m_axis_tready (...),

    // Clock gating status
    .cg_gating          (stream_clk_gated),
    .cg_idle            (stream_idle)
);

// System-level power management
always_ff @(posedge axi_clk) begin
    stream_power_down <= stream_idle && (idle_time >
POWER_DOWN_DELAY);
end
```

See the AXIS4 Clock-Gated Variants Guide for gating and ungating behaviour, including the ungating latency and the `fub_axis_tready` hold-off while gated.

3.7 Design Notes

3.7.1 Area Optimization

- Use minimum required data widths
- Disable unused sideband signals (set width to 0)
- Optimize buffer depths for application requirements
- Share buffers across multiple streams when possible

3.7.2 Timing Optimization

- Register all interface outputs for timing closure
- Use appropriate buffer depths to break critical paths
- Consider pipeline stages for very high-frequency designs

3.7.3 Power Optimization

- Use clock gating variant (`axis4_master_cg`) when available
- Implement activity-based power scaling
- Size buffers appropriately to minimize switching

3.7.4 Known Limitations

Limitation	Detail
No TKEEP	Only TSTRB is implemented. A

Limitation	Detail
	protocol-compliant null-byte / position-byte distinction is not available. See TSTRB and TKEEP
No `TWAKEUP`	The AXI4-Stream `TWAKEUP` low-power signal is not implemented
No native CDC	Both interfaces are on `aclk`. Crossing clock domains requires an external `gaxi_fifo_async`
No reordering or arbitration	The module is a single-stream elastic buffer. `TID`/`TDEST` are carried through unmodified; they are not decoded for routing
No protocol checking	The module does not detect or report `TVALID` deassertion before `TREADY`, or `TLAST` framing errors
`SKID_DEPTH` limited to 2..8 inclusive	The buffer is a timing element, not a rate adapter. Use a `gaxi_fifo_sync` downstream for deep elastic storage

3.7.5 Buffer Depth Selection

Choose buffer depths based on application characteristics:

Legal values are 2..8 inclusive entries. The skid buffer is a *timing* element, not a rate adaptation FIFO — if an application needs tens or hundreds of entries of elastic storage, place a `gaxi_fifo_sync` (or `gaxi_fifo_async` across clock domains) downstream rather than attempting to scale `SKID_DEPTH`.

Application Type	Recommended SKID_DEPTH	Buffer Size	Rationale
Low Latency DSP	2	2 entries	Reduce processing

Application Type	Recommended SKID_DEPTH	Buffer Size	Rationale
			delay
Video Streaming	4	4 entries	Absorb short backpressure bubbles
Network Packets	6	6 entries	Tolerate bursty sink stalls
High Throughput	8	8 entries	Maximum decoupling available

3.7.6 Data Width Optimization

```
// Optimize data width for application
localparam VIDEO_PIXELS_PER_CYCLE = 4;
localparam PIXEL_BITS = 24;  // RGB888
localparam VIDEO_DATA_WIDTH = VIDEO_PIXELS_PER_CYCLE * PIXEL_BITS;

axis4_master #(
    .AXIS_DATA_WIDTH(VIDEO_DATA_WIDTH),  // 96 bits
    .SKID_DEPTH(4),
    .AXIS_USER_WIDTH(8)  // Control signals
) u_video_master (...);
```

3.7.7 Sideband Signal Optimization

```
// Minimize unused sideband signals for area/power
axis4_master #(
    .AXIS_DATA_WIDTH(128),
    .AXIS_ID_WIDTH(0),      // Disable TID
    .AXIS_DEST_WIDTH(0),    // Disable TDEST
    .AXIS_USER_WIDTH(4),    // Minimal TUSER
    .SKID_DEPTH(2)
) u_optimized_master (...);
```

3.8 Related Modules

- **axis4_master_cg**: Clock-gated version for power optimization
- **axis4_slave**: Complementary AXI4-Stream slave implementation
- **gaxi_skid_buffer**: Underlying buffer infrastructure
- **gaxi_fifo_async**: Asynchronous FIFO for clock domain crossing

> The `axis_arbiter` and `axis_interconnect` blocks referenced in the "Multi-Stream Router Integration" example above are **not part of this repository**. That example illustrates a system topology built around `axis4_master`; the arbitration block is left to the integrator.

The `axis4_master` module provides a complete, high-performance solution for AXI4-Stream master functionality with advanced buffering, flexible signal configuration, and comprehensive system integration capabilities.

3.9 Testing

`val/amba/test_axis4_master.py` exercises this module. It collects 14 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis4_master.py -v
```

3.9.1 What to Verify

Protocol Compliance - Verify AXI4-Stream handshaking (VALID/READY) - Check TLAST alignment with packet boundaries - Validate sideband signal preservation

Buffer Verification - Test buffer overflow/underflow protection - Verify data integrity through buffering - Check flow control under backpressure

Performance Verification - Measure sustained throughput under various loads - Verify buffer utilization efficiency - Check latency characteristics

---

3.10 Navigation

- **← Back to axis4 index**
- **← Back to rtl-amba index**

4 axis4_master_cg

Module: `axis4_master_cg.sv` **Base Module:** axis4_master **Location:** `rtl/amba/axis4/` **Status:** Production Ready

---

4.1 Overview

`axis4_master_cg` is the clock-gated variant of `axis4_master`: the same AXI4-Stream transport, wrapped in one `amba_clock_gate_ctrl` that stops the inner module's clock while the stream is idle. Functionally it is indistinguishable from `axis4_master`; what it adds is the gating and the `cg_gating` / `cg_idle` status outputs.

For complete clock-gating documentation — wakeup terms, ungating latency, configuration guidelines — see the AXIS4 Clock-Gated Variants Guide.

4.2 Parameters

All of the axis4_master parameters, plus one clock-gating parameter:

Parameter	Type	Default	Description
`SKID_DEPTH`	int	4	Skid buffer depth in entries (2..8 inclusive), passed directly to `gaxi_skid_buffer.DEPTH`
`AXIS_DATA_WIDTH`	int	32	AXI4-Stream data bus width in bits (must be a multiple of 8)
`AXIS_ID_WIDTH`	int	8	Stream ID width (0 to disable)
`AXIS_DEST_WIDTH`	int	4	Destination width (0 to disable)
`AXIS_USER_WIDTH`	int	1	User signal width (0 to disable)
`CG_IDLE_COUNT_WIDTH`	int	4	Width of the idle countdown counter (max idle = 2^N - 1 cycles)

`CG_IDLE_COUNT_WIDTH` is the **only** additional parameter. Gating enable and the idle threshold are **runtime inputs**, not parameters — they appear in the port list, not here.

> There is no `ENABLE_CLOCK_GATING` parameter, no `CG_IDLE_CYCLES` parameter, and no `CG_GATE_*` domain-enable family. There is a single gating domain covering the whole module. The `_cg` wrapper also **does**

not expose the base module's `busy` output — it is consumed internally as a wakeup term.

The base module's parameters are `SKID_DEPTH`, `AXIS_DATA_WIDTH`, `AXIS_ID_WIDTH`, `AXIS_DEST_WIDTH` and `AXIS_USER_WIDTH`. AXI4-Stream has no address channel, so there is no `AXI_ADDR_WIDTH`.

4.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`DW`	`AXIS_DATA_WIDTH`
`IW`	`AXIS_ID_WIDTH`
`DESTW`	`AXIS_DEST_WIDTH`
`UW`	`AXIS_USER_WIDTH`
`SW`	`DW / 8`
`IW_WIDTH`	`(IW > 0) ? IW : 1`
`DESTW_WIDTH`	`(DESTW > 0) ? DESTW : 1`
`UW_WIDTH`	`(UW > 0) ? UW : 1`

4.3 Ports

4.3.1 Clock and Reset

Port	Width	Direction	Description
`aclk`	1	Input	AXI4-Stream clock
`aresetn`	1	Input	AXI4-Stream active-low reset

4.3.2 Clock Gating Configuration

Port	Width	Direction	Description
`cfg_cg_enable`	1	Input	Enable clock gating (0 = clock always running)

Port	Width	Direction	Description
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating engages

4.3.3 Slave AXI4-Stream Interface (Input Side - FUB)

Port	Width	Direction	Description
fub_axis_tdata	DW	Input	Stream data
fub_axis_tstrb	SW	Input	Data byte strobes
fub_axis_tlast	1	Input	Last transfer in packet
fub_axis_tid	IW_WIDTH	Input	Stream identifier
fub_axis_tdest	DESTW_WIDTH	Input	Destination routing
fub_axis_tuser	UW_WIDTH	Input	User-defined sideband
fub_axis_tvalid	1	Input	Transfer valid
fub_axis_tready	1	Output	Transfer ready

4.3.4 Master AXI4-Stream Interface (Output Side)

Port	Width	Direction	Description
m_axis_tdata	DW	Output	Stream data
m_axis_tstrb	SW	Output	Data byte strobes
m_axis_tlast	1	Output	Last transfer in packet
m_axis_tid	IW_WIDTH	Output	Stream identifier
m_axis_tdest	DESTW_WIDTH	Output	Destination routing
m_axis_tuser	UW_WIDTH	Output	User-defined sideband

Port	Width	Direction	Description
m_axis_tvalid	1	Output	Transfer valid
m_axis_tready	1	Input	Transfer ready

4.3.5 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	Clock currently gated (1=gated, 0=running)
cg_idle	1	Output	No activity observed in the previous cycle

> The `_cg` wrappers do not expose `busy`. The base module's `busy` output is consumed internally as one of the wakeup terms and is not brought out. Use `cg_idle` for system-level power sequencing instead. Apart from that substitution, and the two `cfg_cg_*` inputs above, the port list matches the base module.

4.4 Functional Description

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to the base module and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (this side's valids plus the base module's `busy`) and `axi_valid` (the far side's valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `!cg_gating`, so a peer sees no acceptance until the clock runs again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

4.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH	4 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

4.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
axis4_master_cg #(
    .SKID_DEPTH            (4),
    .AXIS_DATA_WIDTH       (32),
    .AXIS_ID_WIDTH         (8),
    .AXIS_DEST_WIDTH       (4),
    .AXIS_USER_WIDTH       (1),
    .CG_IDLE_COUNT_WIDTH   (4),
    .DW                    (AXIS_DATA_WIDTH),
    .IW                    (AXIS_ID_WIDTH),
    .DESTW                 (AXIS_DEST_WIDTH),
    .UW                    (AXIS_USER_WIDTH)
) u_axis_master_cg (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .cfg_cg_enable         (cfg_cg_enable),
    .cfg_cg_idle_count     (cfg_cg_idle_count),
    .fub_axis_tdata        (fub_axis_tdata),
    .fub_axis_tstrb        (fub_axis_tstrb),
    .fub_axis_tlast        (fub_axis_tlast),
    .fub_axis_tid          (fub_axis_tid),
    .fub_axis_tdest        (fub_axis_tdest),
    .fub_axis_tuser        (fub_axis_tuser),
    .fub_axis_tvalid       (fub_axis_tvalid),
    .fub_axis_tready       (fub_axis_tready),
    .m_axis_tdata          (m_axis_tdata),
    .m_axis_tstrb          (m_axis_tstrb),
    .m_axis_tlast          (m_axis_tlast),
    .m_axis_tid            (m_axis_tid),
    .m_axis_tdest          (m_axis_tdest),
    .m_axis_tuser          (m_axis_tuser),
    .m_axis_tvalid         (m_axis_tvalid),
```

```
    .m_axis_tready          (m_axis_tready),
    .cg_gating              (cg_gating),
    .cg_idle                (cg_idle)
);
```

4.6.1 Quick Usage

```
axis4_master_cg #(
    // Base module parameters (see axis4_master.md)
    .SKID_DEPTH(4),
    .AXIS_DATA_WIDTH(64),
    .AXIS_ID_WIDTH(8),
    .AXIS_DEST_WIDTH(4),
    .AXIS_USER_WIDTH(1),

    // Clock gating (see CG guide for details)
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Clock gating control (runtime inputs)
    .cfg_cg_enable(cg_enable),
    .cfg_cg_idle_count(4'd8),

    // ... all AXI4-Stream ports same as axis4_master ...

    // Clock gating status (replaces the base module's `busy` output)
    .cg_gating(clk_is_gated),
    .cg_idle(stream_idle)
);
```

4.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

4.7.1 Summary

The `axis4_master_cg` module adds power optimization to `axis4_master` through activity-based clock gating:

- **Same Data Functionality:** Identical to the base module once the clock is running
- **Power Savings:** Estimated 25-70% depending on stream duty cycle (planning figure, not measured)
- **Configurable:** Runtime idle threshold and enable via `cfg_cg_*` inputs
- **Bypass When Disabled:** `cfg_cg_enable = 0` holds the clock permanently enabled, making the wrapper functionally identical to the base module

---

4.8 Related Modules

Read out of the RTL, not curated: these are the modules this one instantiates and the modules that instantiate it.

Instantiates: - `amba_clock_gate_ctrl` - `axis4_master`

4.8.1 Documentation

- **Base Module Functionality:** axis4_master.md
- **Clock Gating Guide:** axis4_clock_gating_guide.md (AXIS4-specific)
- **Generic CG Architecture:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

4.9 Testing

`val/amba/test_axis4_master_cg.py` exercises this module. It collects 2 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis4_master_cg.py -v
```

4.10 Navigation

- ← Back to AXIS4 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

5 axis4_slave

An AXI4-Stream slave module that provides high-throughput streaming data reception with configurable buffering and comprehensive support for all AXI4-Stream sideband signals including ID, DEST, and USER channels.

5.1 Overview

The `axis4_slave` module implements a complete AXI4-Stream slave interface with integrated skid buffering for optimal streaming performance. It supports the full AXI4-Stream protocol with configurable data widths, optional sideband signals, and intelligent buffer management for streaming data applications. It's the mirror image of `axis4_master` — the interconnect talks to the `s_axis_*` side, your backend drinks from the `fub_axis_*` side, and the skid buffer in the middle keeps the two from tripping over each other.

5.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Skid buffer depth in **entries** (not a log2 exponent). Passed directly to `gaxi_skid_buffer.DEPTH`, which supports 2..8 inclusive
AXIS_DATA_WIDTH	int	32	AXI4-Stream data bus width in **bits** (must be a multiple of 8; `SW = AXIS_DATA_WIDTH/8`)
AXIS_ID_WIDTH	int	8	Stream ID width (0 to

Parameter	Type	Default	Description
			disable)
AXIS_DEST_WIDTH	int	4	Destination width (0 to disable)
AXIS_USER_WIDTH	int	1	User signal width (0 to disable)

> **SKID_DEPTH is a literal entry count.** `SKID_DEPTH = 4` yields a 4-entry buffer, not 16. The underlying `gaxi_skid_buffer` is a shift-register FIFO whose `count` port is 4 bits wide; only the values 2..8 inclusive are supported. Any integer in that range is legal, odd values included – `gaxi_skid_buffer` states “2..8 inclusive (any integer)” and guards it. The `{2,4,6,8}` restriction claimed here was an inference, not a contract.

5.3 Ports

The full declaration, straight from the RTL:

```
module axis4_slave #(
    parameter int SKID_DEPTH         = 4,
    parameter int AXIS_DATA_WIDTH    = 32,
    parameter int AXIS_ID_WIDTH      = 8,
    parameter int AXIS_DEST_WIDTH    = 4,
    parameter int AXIS_USER_WIDTH    = 1,

    // Short and calculated params
    parameter int DW       = AXIS_DATA_WIDTH,
    parameter int IW       = AXIS_ID_WIDTH,
    parameter int DESTW    = AXIS_DEST_WIDTH,
    parameter int UW       = AXIS_USER_WIDTH,
    parameter int SW       = DW / 8,
    parameter int IW_WIDTH = (IW > 0) ? IW : 1,
    parameter int DESTW_WIDTH = (DESTW > 0) ? DESTW : 1,
    parameter int UW_WIDTH = (UW > 0) ? UW : 1,
    parameter int TSize    = DW+SW+1+IW_WIDTH+DESTW_WIDTH+UW_WIDTH
) (
    // Global Clock and Reset
    input  logic                       aclk,
    input  logic                       aresetn,

    // Slave AXI4-Stream Interface (Input Side from interconnect)
    input  logic [DW-1:0]              s_axis_tdata,
    input  logic [SW-1:0]              s_axis_tstrb,
    input  logic                       s_axis_tlast,
```

```
    input  logic [IW_WIDTH-1:0]         s_axis_tid,
    input  logic [DESTW_WIDTH-1:0]      s_axis_tdest,
    input  logic [UW_WIDTH-1:0]         s_axis_tuser,
    input  logic                        s_axis_tvalid,
    output logic                        s_axis_tready,

    // Master AXI4-Stream Interface (Output Side to backend)
    output logic [DW-1:0]               fub_axis_tdata,
    output logic [SW-1:0]               fub_axis_tstrb,
    output logic                        fub_axis_tlast,
    output logic [IW_WIDTH-1:0]         fub_axis_tid,
    output logic [DESTW_WIDTH-1:0]      fub_axis_tdest,
    output logic [UW_WIDTH-1:0]         fub_axis_tuser,
    output logic                        fub_axis_tvalid,
    input  logic                        fub_axis_tready,

    // Status outputs for clock gating
    output logic                        busy
);
```

5.3.1 Slave AXI4-Stream Interface (Input from Interconnect)

Port	Width	Direction	Description
s_axis_tdata	DW	Input	Stream data
s_axis_tstrb	SW	Input	Data strobes (byte-valid indicators)
s_axis_tlast	1	Input	Last transfer in packet
s_axis_tid	IW	Input	Stream ID (routing/reordering)
s_axis_tdest	DESTW	Input	Destination routing
s_axis_tuser	UW	Input	User-defined sideband
s_axis_tvalid	1	Input	Data valid
s_axis_tready	1	Output	Ready to accept data

5.3.2 Backend Interface (Output to Processing Logic)

Port	Width	Direction	Description
fub_axis_tdata	DW	Output	Stream data (to backend)
fub_axis_tstrb	SW	Output	Data strobes
fub_axis_tlast	1	Output	Last transfer in packet
fub_axis_tid	IW	Output	Stream ID
fub_axis_tdest	DESTW	Output	Destination routing
fub_axis_tuser	UW	Output	User-defined sideband
fub_axis_tvalid	1	Output	Data valid (to backend)
fub_axis_tready	1	Input	Backend ready

5.3.3 Status

Port	Width	Direction	Description
busy	1	Output	Interface active (for clock gating)

5.4 Functional Description

The slave accepts an AXI4-Stream on `s_axis_*` and presents it on the FUB-facing `fub_axis_*` side through a `gaxi_skid_buffer`. The buffer registers both `rd_valid` and its storage, so every beat costs one cycle even when neither side stalls; `SKID_DEPTH` sets how much upstream backpressure can be absorbed before it propagates, not the sustained rate.

TLAST, TSTRB, TID, TDEST and TUSER are carried verbatim when their widths are non-zero. The module holds no packet state – it is a buffer, not a framer.

5.4.1 Busy Signal Generation

```
busy = (buffer_count > 0) || s_axis_tvalid;
```

`busy` is asserted while any beat is held in the skid buffer or a new beat is being offered on the slave interface. It is the wakeup term consumed by `axis4_slave_cg`.

5.5 Timing Characteristics

Characteristic	Value	Description
Buffer Depth	4 entries (default)	`SKID_DEPTH` entries, verbatim
Buffering Latency	1 clock cycle	`s_axis` to `fub_axis` delay
Flow Control Latency	1 clock cycle	`fub_axis_tready` to `s_axis_tready` propagation
Sustained Throughput	1 beat/cycle	Once the pipeline is primed

`gaxi_skid_buffer` registers both `rd_valid` and the storage array, so the 1-cycle latency applies on **every** transfer, including the unstalled case. There is no combinational path from `s_axis_tdata` to `fub_axis_tdata`.

5.6 Usage Examples

```
axis4_slave #(
    .SKID_DEPTH(4),          // 4 entries
    .AXIS_DATA_WIDTH(64),
    .AXIS_ID_WIDTH(8),
    .AXIS_DEST_WIDTH(4),
    .AXIS_USER_WIDTH(1)
) u_axis_slave (
    .aclk(stream_clk),
    .aresetn(stream_resetn),

    // Slave interface (from network/interconnect)
    .s_axis_tdata(net_tdata),
    .s_axis_tstrb(net_tstrb),
    .s_axis_tlast(net_tlast),
    .s_axis_tid(net_tid),
    .s_axis_tdest(net_tdest),
    .s_axis_tuser(net_tuser),
    .s_axis_tvalid(net_tvalid),
    .s_axis_tready(net_tready),

    // Backend interface (to processing logic)
    .fub_axis_tdata(proc_tdata),
    .fub_axis_tstrb(proc_tstrb),
    .fub_axis_tlast(proc_tlast),
    .fub_axis_tid(proc_tid),
```

```
    .fub_axis_tdest(proc_tdest),
    .fub_axis_tuser(proc_tuser),
    .fub_axis_tvalid(proc_tvalid),
    .fub_axis_tready(proc_tready),

    .busy(slave_busy)
);
```

5.7 Design Notes

- **Signal Flow:** Interconnect → Skid Buffer → Backend
- **Use Case:** Stream receivers, packet processors, video decoders
- **Buffering:** Allows backend to consume at its own pace without stalling interconnect

5.7.1 Features

- **Full AXI4-Stream Protocol:** Support for all optional sideband signals
- **Elastic Buffering:** Skid buffer decouples interconnect and backend timing
- **Configurable Width:** Optional ID, DEST, USER signals (set width to 0 to disable)
- **Activity Monitoring:** Busy signal for power management

5.7.2 Known Limitations

Limitation	Detail
No `TKEEP`	Only `TSTRB` is implemented. A protocol-compliant null-byte / position-byte distinction is not available
No `TWAKEUP`	The AXI4-Stream `TWAKEUP` low-power signal is not implemented
No native CDC	Both interfaces are on `aclk`. Crossing clock domains requires an external `gaxi_fifo_async`
No routing or demultiplexing	`TID`/`TDEST` are carried through unmodified; they are not decoded
No protocol checking	The module does not detect or

Limitation	Detail
	report `TVALID` deassertion before `TREADY`, or `TLAST` framing errors
`SKID_DEPTH` limited to 2..8 inclusive	The buffer is a timing element, not a rate adapter. Use a `gaxi_fifo_sync` downstream for deep elastic storage

5.7.3 TSTRB and TKEEP

ARM IHI 0051A defines two byte qualifiers: TKEEP (byte is part of the data stream) and TSTRB (byte is a data byte rather than a position byte). This module implements **TSTRB only** — there is no TKEEP port. `s_axis_tstrb` is packed into the skid buffer entry and reproduced on `fub_axis_tstrb` unmodified, so it can equally carry a TKEEP mask if the surrounding design treats it that way. That is an integrator-side naming convention, not protocol-compliant TKEEP support.

5.8 Related Modules

- **axis4_master** - Stream master counterpart
- **axis4_slave_cg** - Clock-gated version
- **AXIS4 Clock-Gated Variants Guide** - Clock gating configuration and behaviour

---

Last Updated: 2025-10-20

5.9 Testing

`val/amba/test_axis4_slave.py` exercises this module. It collects 14 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/amba/test_axis4_slave.py -v
```

5.10 Navigation

- ← Back to AXIS4 Index
- ← Back to rtl-amba Index

6 axis4_slave_cg

Module: `axis4_slave_cg.sv` **Base Module:** axis4_slave **Location:** `rtl/amba/axis4/` **Status:** Production Ready

---

6.1 Overview

`axis4_slave_cg` is the clock-gated variant of `axis4_slave`: the same AXI4-Stream transport, wrapped in one `amba_clock_gate_ctrl` that stops the inner module's clock while the stream is idle. Functionally it is indistinguishable from `axis4_slave`; what it adds is the gating and the `cg_gating` / `cg_idle` status outputs.

For complete clock-gating documentation — wakeup terms, ungating latency, configuration guidelines — see the AXIS4 Clock-Gated Variants Guide.

6.2 Parameters

All of the axis4_slave parameters, plus one clock-gating parameter:

Parameter	Type	Default	Description
`SKID_DEPTH`	int	4	Skid buffer depth in entries (2..8 inclusive), passed directly to `gaxi_skid_buffer.DEPTH`
`AXIS_DATA_WIDTH`	int	32	AXI4-Stream data bus width in bits (must be a multiple of 8)
`AXIS_ID_WIDTH`	int	8	Stream ID width (0 to disable)
`AXIS_DEST_WIDTH`	int	4	Destination width (0 to disable)
`AXIS_USER_WIDTH`	int	1	User signal width (0 to disable)
`CG_IDLE_COUNT_WIDTH`	int	4	Width of the idle countdown counter (max

Parameter	Type	Default	Description
			idle = 2^N - 1 cycles)

`CG_IDLE_COUNT_WIDTH` is the **only** additional parameter. Gating enable and the idle threshold are **runtime inputs**, not parameters — they appear in the port list, not here.

> There is no `ENABLE_CLOCK_GATING` parameter, no `CG_IDLE_CYCLES` parameter, and no `CG_GATE_*` domain-enable family. There is a single gating domain covering the whole module. The `_cg` wrapper also **does not expose the base module's `busy` output** — it is consumed internally as a wakeup term.
>
> The base module's parameters are `SKID_DEPTH`, `AXIS_DATA_WIDTH`, `AXIS_ID_WIDTH`, `AXIS_DEST_WIDTH` and `AXIS_USER_WIDTH`. AXI4-Stream has no address channel, so there is no `AXI_ADDR_WIDTH`.

6.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`DW`	`AXIS_DATA_WIDTH`
`IW`	`AXIS_ID_WIDTH`
`DESTW`	`AXIS_DEST_WIDTH`
`UW`	`AXIS_USER_WIDTH`
`SW`	`DW / 8`
`IW_WIDTH`	`(IW > 0) ? IW : 1`
`DESTW_WIDTH`	`(DESTW > 0) ? DESTW : 1`
`UW_WIDTH`	`(UW > 0) ? UW : 1`

6.3 Ports

6.3.1 Clock and Reset

Port	Width	Direction	Description
`aclk`	1	Input	AXI4-Stream clock
`aresetn`	1	Input	AXI4-Stream

Port	Width	Direction	Description
			active-low reset

6.3.2 Clock Gating Configuration

Port	Width	Direction	Description
`cfg_cg_enable`	1	Input	Enable clock gating (0 = clock always running)
`cfg_cg_idle_count`	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating engages

6.3.3 Slave AXI4-Stream Interface (Input from Interconnect)

Port	Width	Direction	Description
`s_axis_tdata`	DW	Input	Stream data
`s_axis_tstrb`	SW	Input	Data strobes (byte-valid indicators)
`s_axis_tlast`	1	Input	Last transfer in packet
`s_axis_tid`	IW_WIDTH	Input	Stream ID (routing/reordering)
`s_axis_tdest`	DESTW_WIDTH	Input	Destination routing
`s_axis_tuser`	UW_WIDTH	Input	User-defined sideband
`s_axis_tvalid`	1	Input	Data valid
`s_axis_tready`	1	Output	Ready to accept data

6.3.4 Backend Interface (Output to Processing Logic)

Port	Width	Direction	Description
`fub_axis_tdata`	DW	Output	Stream data (to backend)

Port	Width	Direction	Description
fub_axis_tstrb	SW	Output	Data strobes
fub_axis_tlast	1	Output	Last transfer in packet
fub_axis_tid	IW_WIDTH	Output	Stream ID
fub_axis_tdest	DESTW_WIDTH	Output	Destination routing
fub_axis_tuser	UW_WIDTH	Output	User-defined sideband
fub_axis_tvalid	1	Output	Data valid (to backend)
fub_axis_tready	1	Input	Backend ready

6.3.5 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	Clock currently gated (1=gated, 0=running)
cg_idle	1	Output	No activity observed in the previous cycle

> The `_cg` wrappers do not expose `busy`. The base module's `busy` output is consumed internally as one of the wakeup terms and is not brought out. Use `cg_idle` for system-level power sequencing instead. Apart from that substitution, and the two `cfg_cg_*` inputs above, the port list matches the base module.

6.4 Functional Description

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to the base module and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (this side's valids plus the base module's `busy`) and `axi_valid` (the far side's valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `!cg_gating`, so a peer sees no acceptance until the clock runs again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

6.5 Timing Characteristics

Skid parameter	Default depth
`SKID_DEPTH`	4 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

6.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
axis4_slave_cg #(
    .SKID_DEPTH            (4),
    .AXIS_DATA_WIDTH       (32),
    .AXIS_ID_WIDTH         (8),
    .AXIS_DEST_WIDTH       (4),
    .AXIS_USER_WIDTH       (1),
    .CG_IDLE_COUNT_WIDTH   (4),
    .DW                    (AXIS_DATA_WIDTH),
    .IW                    (AXIS_ID_WIDTH),
    .DESTW                 (AXIS_DEST_WIDTH),
    .UW                    (AXIS_USER_WIDTH)
) u_axis_slave_cg (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .cfg_cg_enable         (cfg_cg_enable),
    .cfg_cg_idle_count     (cfg_cg_idle_count),
    .s_axis_tdata          (s_axis_tdata),
```

```
    .s_axis_tstrb          (s_axis_tstrb),
    .s_axis_tlast          (s_axis_tlast),
    .s_axis_tid            (s_axis_tid),
    .s_axis_tdest          (s_axis_tdest),
    .s_axis_tuser          (s_axis_tuser),
    .s_axis_tvalid         (s_axis_tvalid),
    .s_axis_tready         (s_axis_tready),
    .fub_axis_tdata        (fub_axis_tdata),
    .fub_axis_tstrb        (fub_axis_tstrb),
    .fub_axis_tlast        (fub_axis_tlast),
    .fub_axis_tid          (fub_axis_tid),
    .fub_axis_tdest        (fub_axis_tdest),
    .fub_axis_tuser        (fub_axis_tuser),
    .fub_axis_tvalid       (fub_axis_tvalid),
    .fub_axis_tready       (fub_axis_tready),
    .cg_gating             (cg_gating),
    .cg_idle               (cg_idle)
);
```

6.6.1 Quick Usage

```
axis4_slave_cg #(
    // Base module parameters (see axis4_slave.md)
    .SKID_DEPTH(4),
    .AXIS_DATA_WIDTH(64),
    .AXIS_ID_WIDTH(8),
    .AXIS_DEST_WIDTH(4),
    .AXIS_USER_WIDTH(1),

    // Clock gating (see CG guide for details)
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Clock gating control (runtime inputs)
    .cfg_cg_enable(cg_enable),
    .cfg_cg_idle_count(4'd8),

    // ... all AXI4-Stream ports same as axis4_slave ...

    // Clock gating status (replaces the base module's `busy` output)
    .cg_gating(clk_is_gated),
    .cg_idle(stream_idle)
);
```

6.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

6.7.1 Summary

The `axis4_slave_cg` module adds power optimization to `axis4_slave` through activity-based clock gating:

- **Same Data Functionality:** Identical to the base module once the clock is running
- **Power Savings:** Estimated 25-70% depending on stream duty cycle (planning figure, not measured)
- **Configurable:** Runtime idle threshold and enable via `cfg_cg_*` inputs
- **Bypass When Disabled:** `cfg_cg_enable = 0` holds the clock permanently enabled, making the wrapper functionally identical to the base module

---

6.8 Related Modules

Read out of the RTL, not curated: these are the modules this one instantiates and the modules that instantiate it.

Instantiates: - `amba_clock_gate_ctrl` - `axis4_slave`

6.8.1 Documentation

- **Base Module Functionality:** axis4_slave.md

- **Clock Gating Guide:** axis4_clock_gating_guide.md (AXIS4-specific)
- **Generic CG Architecture:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

6.9 Testing

`val/amba/test_axis4_slave_cg.py` exercises this module. It collects 2 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis4_slave_cg.py -v
```

6.10 Navigation

- **← Back to AXIS4 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**